

CS224N Lecture 7: Attention Mechanisms and Neural Machine Translation Evaluation

Stanford University

Spring 2024 – Week 4

Abstract

This lecture introduces one of the most transformative concepts in modern neural networks: the **attention mechanism**. We begin by discussing evaluation metrics for machine translation (BLEU score), review the progress in neural MT, and then dive deep into attention—how it works, why it’s revolutionary, and different architectural variants. Attention addresses the bottleneck problem in sequence-to-sequence models and has become a fundamental building block of modern NLP architectures, particularly Transformers.

Contents

1	Machine Translation Evaluation	2
1.1	The Challenge of Evaluating Translation	2
1.2	BLEU Score	2
1.2.1	Historical Context	2
1.2.2	BLEU Methodology	2
1.2.3	BLEU Formula	3
1.3	BLEU Score Interpretation	3
1.4	Limitations of BLEU	3
2	Progress in Machine Translation	4
2.1	Historical Timeline	4
2.2	Statistical MT (SMT)	4
2.3	Neural MT Revolution	5
3	The Bottleneck Problem in Seq2Seq	5
3.1	Problem Statement	5
3.2	Issues with Fixed Context Vector	5
4	Attention Mechanism	6
4.1	Core Intuition	6
4.2	Attention Architecture Overview	6
4.3	Attention Mechanism: Step-by-Step	6
4.3.1	Step 1: Compute Attention Scores	6
4.3.2	Step 2: Compute Attention Distribution	7
4.3.3	Step 3: Compute Attention Output (Context Vector)	7
4.3.4	Step 4: Concatenate and Generate Output	7
4.4	Complete Attention Equations	8

5	Attention Score Functions	8
5.1	Dot Product Attention	8
5.2	Multiplicative (Bilinear) Attention	9
5.3	Scaled Dot Product Attention	9
5.4	Reduced-Rank Multiplicative Attention	9
5.5	Additive Attention	10
5.6	Comparison of Attention Variants	10
6	Benefits of Attention	11
6.1	1. Improved NMT Performance	11
6.2	2. Solves Bottleneck Problem	11
6.3	3. Helps Vanishing Gradient	11
6.4	4. More Human-Like	11
6.5	5. Provides Interpretability	11
7	Attention as a General Tool	12
7.1	Beyond Machine Translation	12
7.2	Applications of Attention	12
7.3	Why Attention Works So Well	13
8	Summary and Key Takeaways	14
8.1	Historical Impact	14
9	Further Reading	14
9.1	Foundational Papers	14
9.2	Additional Resources	15

1 Machine Translation Evaluation

1.1 The Challenge of Evaluating Translation

Unlike classification tasks where there is typically one correct answer, **translation has no single correct solution**. Any given sentence can be translated in multiple valid ways, with different:

- Word choices (synonyms)
- Sentence structures (syntax)
- Word orderings
- Levels of formality

Example 1.1 (Translation Variability). The French sentence “Il fait beau aujourd’hui” can be translated as:

- “It is nice today”
- “The weather is nice today”
- “It’s beautiful today”
- “Today is a beautiful day”

All are valid translations with slightly different nuances.

1.2 BLEU Score

Definition 1.1 (BLEU). **BLEU (Bilingual Evaluation Understudy)** is an automatic metric for evaluating machine translation quality by measuring n-gram overlap between a candidate translation and one or more reference translations.

1.2.1 Historical Context

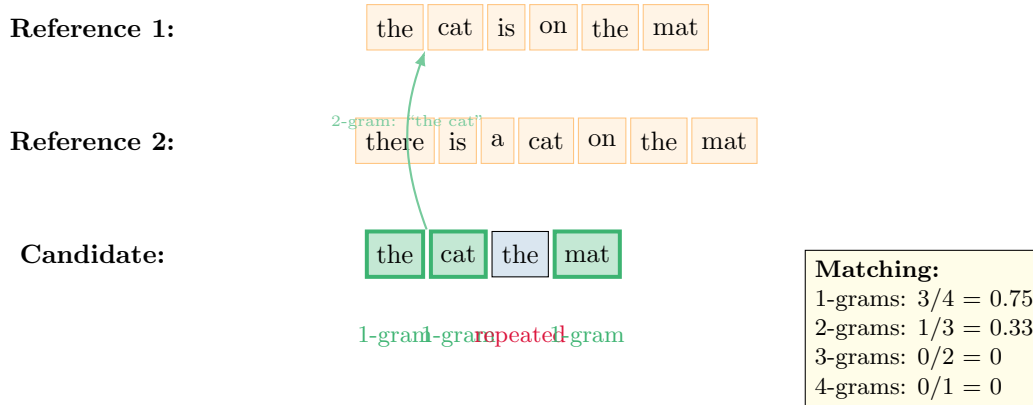
- Proposed by IBM researchers in 2002
- First widely-adopted automatic MT evaluation metric
- Before BLEU: Only human evaluation was used
- Enabled rapid iteration and automatic tuning

1.2.2 BLEU Methodology

BLEU Core Idea

Score translations based on n-gram overlap with reference translations:

1. Count matching n-grams ($n = 1, 2, 3, 4$) between candidate and reference(s)
2. Compute precision for each n-gram size
3. Combine using geometric mean
4. Apply brevity penalty for short translations



1.2.3 BLEU Formula

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (1)$$

where:

- p_n = precision for n-grams of length n
- w_n = uniform weights (typically $1/N$, where $N = 4$)
- BP = brevity penalty to discourage short translations

Brevity Penalty:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases} \quad (2)$$

where c = candidate length, r = reference length.

1.3 BLEU Score Interpretation

BLEU Range	Translation Quality
0–10	Almost unusable
10–20	Gist is clear, many errors
20–30	Understandable, significant errors
30–40	Good translations
40–50	Very good translations
50–60	High quality translations
60+	Near human-level quality

Table 1: Approximate BLEU score interpretation

Remark 1.1. BLEU scores rarely reach 100 due to translation variability. Even human translations typically score 60–70 against other human references.

1.4 Limitations of BLEU

1. **Doesn't consider meaning:** Can reward word overlap without semantic correctness
2. **No handling of synonyms:** “big” vs. “large” treated as completely different
3. **Ignores word order (partially):** Only captures order via n-grams

4. **Reference dependence:** Quality depends heavily on reference translations
5. **Precision-focused:** Doesn't directly measure recall/coverage

Example 1.2 (BLEU Failure Case). **Reference:** “The cat sat on the mat”

Candidate 1: “The the the the the the” (all 1-grams match, but nonsense)

Candidate 2: “A feline rested upon a rug” (perfect meaning, low BLEU)

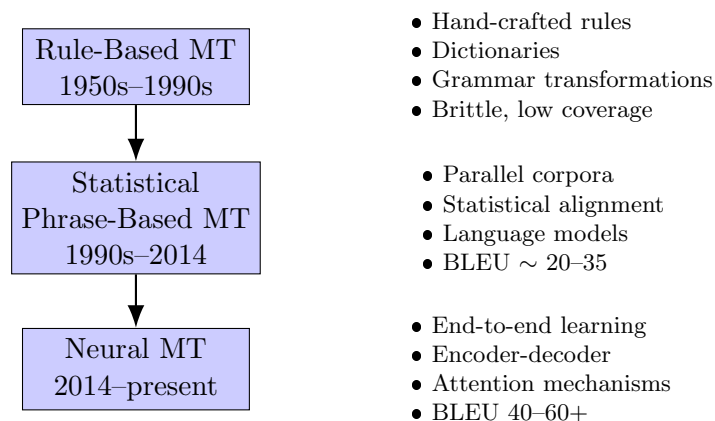
Why BLEU Still Dominates

Despite limitations:

- Fast and automatic
- Correlates reasonably with human judgments at corpus level
- Enables rapid experimentation
- Standard for comparison across papers
- Used in Assignment 3

2 Progress in Machine Translation

2.1 Historical Timeline



2.2 Statistical MT (SMT)

Noisy Channel Model

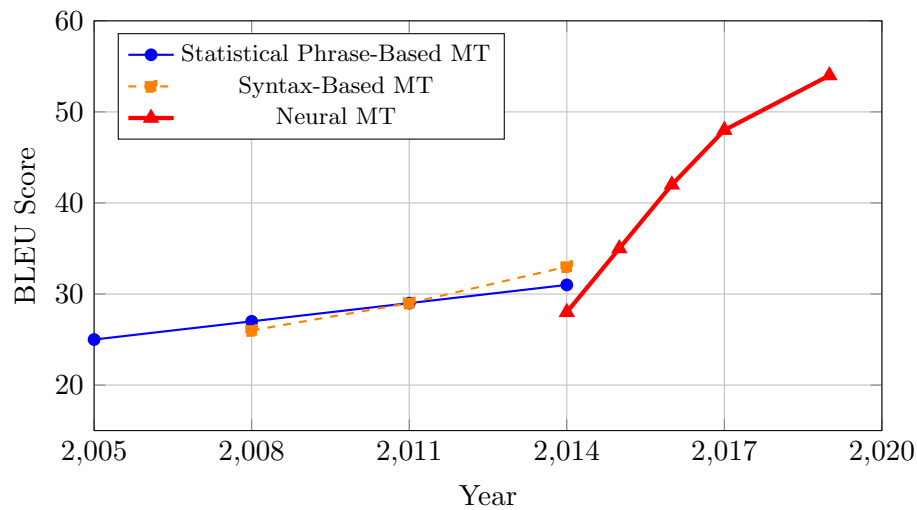
Statistical MT decomposes translation using Bayes' rule:

$$\hat{y} = \arg \max_y P(y | x) = \arg \max_y P(x | y) \cdot P(y) \quad (3)$$

- $P(y)$: **Language model** (target fluency)
- $P(x | y)$: **Translation model** (word correspondences)

Key insight: Separate fluency from adequacy.

2.3 Neural MT Revolution



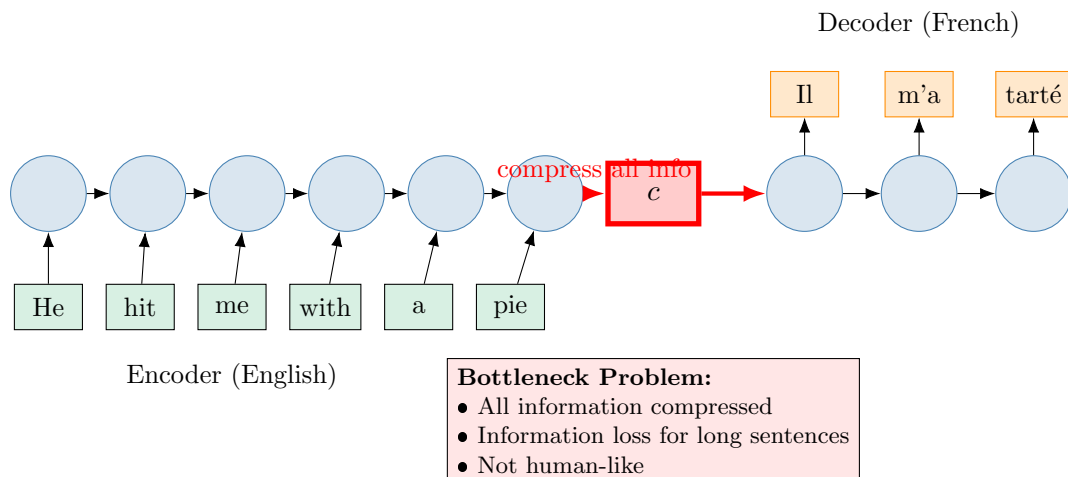
Key Milestones

- **2005–2014:** SMT progress plateaus
- **2014:** First seq2seq NMT (Sutskever et al., Google)
- **2015:** Attention mechanism (Bahdanau et al.)
- **2016:** NMT surpasses SMT; Google deploys NMT
- **2017–present:** Transformer-based NMT dominates

3 The Bottleneck Problem in Seq2Seq

3.1 Problem Statement

In vanilla sequence-to-sequence models, the encoder must compress the *entire source sentence* into a single fixed-size context vector.



3.2 Issues with Fixed Context Vector

1. **Information bottleneck:** One vector must encode entire sentence
2. **Long sentence problem:** Information loss increases with length
3. **Not cognitively plausible:** Humans look back at source while translating

4. **Vanishing gradient:** Harder to learn dependencies from early positions
5. **No word alignment:** No explicit mapping between source and target

Solution: Attention Mechanism

Instead of compressing everything into one vector, allow the decoder to **look back** at all encoder states and **selectively focus** on relevant parts of the source at each decoding step.

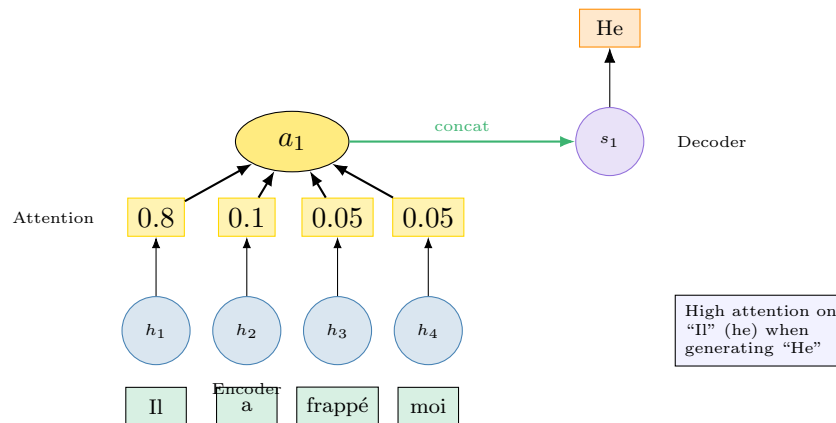
4 Attention Mechanism

4.1 Core Intuition

“On each step of the decoder, we use a direct connection to the encoder to focus on a particular part of the source sequence.”

Key Idea: The decoder maintains a **query** (its current state) and can **attend** to different **values** (encoder states) by computing attention scores.

4.2 Attention Architecture Overview

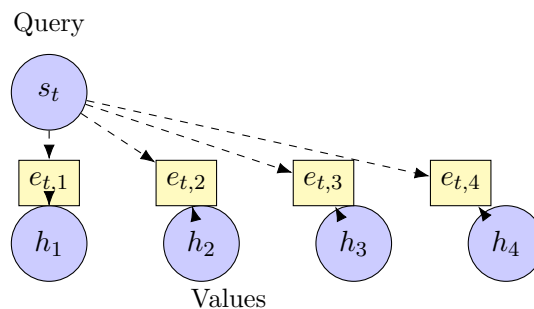


4.3 Attention Mechanism: Step-by-Step

4.3.1 Step 1: Compute Attention Scores

For each encoder hidden state h_i and current decoder state s_t , compute an attention score:

$$e_{t,i} = \text{score}(s_t, h_i) \quad (4)$$

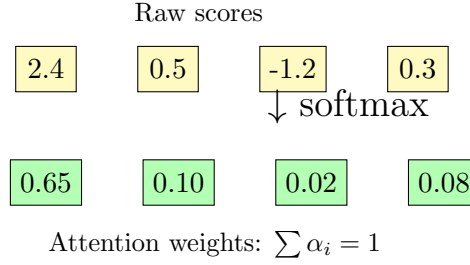


4.3.2 Step 2: Compute Attention Distribution

Apply softmax to get a probability distribution:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})} \quad (5)$$

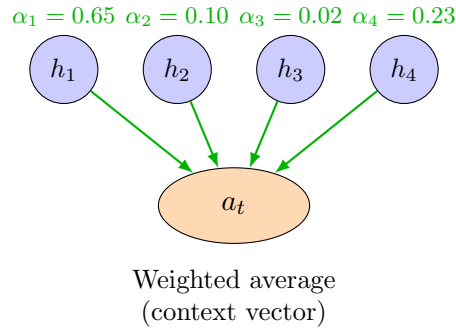
where $\alpha_{t,i}$ is the attention weight on encoder state i at decoder time t .



4.3.3 Step 3: Compute Attention Output (Context Vector)

Weighted sum of encoder states:

$$a_t = \sum_{i=1}^n \alpha_{t,i} \cdot h_i \quad (6)$$

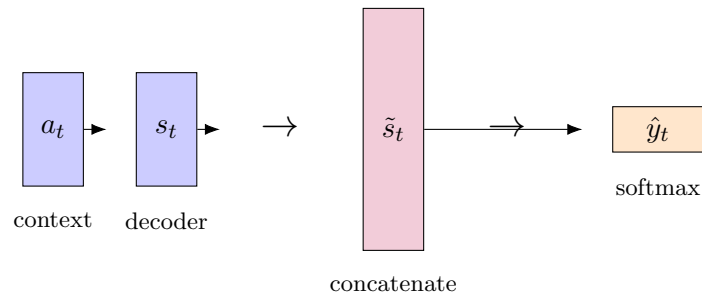


4.3.4 Step 4: Concatenate and Generate Output

Combine attention output with decoder state:

$$\tilde{s}_t = [a_t; s_t] \quad (\text{concatenation}) \quad (7)$$

$$P(y_t \mid y_{<t}, x) = \text{softmax}(W \cdot \tilde{s}_t + b) \quad (8)$$



4.4 Complete Attention Equations

Attention Mechanism (Complete)

Given:

- Encoder hidden states: $h_1, h_2, \dots, h_n$
- Current decoder state: s_t

Compute:

$$e_{t,i} = \text{score}(s_t, h_i) \quad (\text{attention scores}) \quad (9)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})} \quad (\text{attention distribution}) \quad (10)$$

$$a_t = \sum_{i=1}^n \alpha_{t,i} h_i \quad (\text{context vector}) \quad (11)$$

$$\tilde{s}_t = \tanh(W_c[a_t; s_t]) \quad (\text{combined output}) \quad (12)$$

$$P(y_t) = \text{softmax}(W_y \tilde{s}_t) \quad (\text{output distribution}) \quad (13)$$

5 Attention Score Functions

The key architectural choice in attention is **how to compute the score** between query s_t and value h_i .

5.1 Dot Product Attention

Definition 5.1 (Dot Product Attention). The simplest form: compute similarity via dot product.

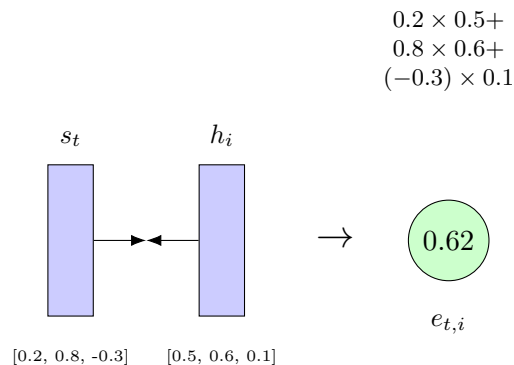
$$\text{score}(s_t, h_i) = s_t^T h_i \quad (14)$$

Advantages:

- Extremely fast (just vector dot product)
- No additional parameters

Disadvantages:

- Requires s_t and h_i to have same dimensionality
- Assumes dimensions correspond semantically
- Limited expressiveness

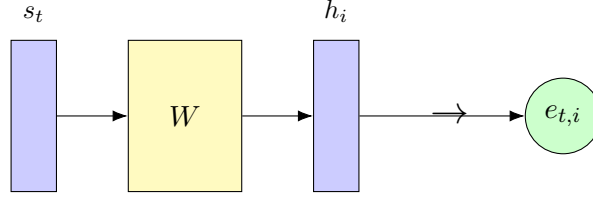


5.2 Multiplicative (Bilinear) Attention

Definition 5.2 (Multiplicative/Bilinear Attention). Introduce a learned weight matrix to allow dimension mismatches and semantic alignment:

$$\text{score}(s_t, h_i) = s_t^T W h_i \quad (15)$$

where $W \in \mathbb{R}^{d_s \times d_h}$ is learned.



Learned weight matrix
enables semantic matching

Advantages:

- Handles different dimensionalities: $s_t \in \mathbb{R}^{d_s}, h_i \in \mathbb{R}^{d_h}$
- Learns which dimensions should align
- More expressive than dot product

Disadvantages:

- Requires $d_s \times d_h$ parameters (can be large)
- Slower than dot product

5.3 Scaled Dot Product Attention

Definition 5.3 (Scaled Dot Product). Scale the dot product to prevent softmax saturation:

$$\text{score}(s_t, h_i) = \frac{s_t^T h_i}{\sqrt{d}} \quad (16)$$

where d is the dimensionality.

Motivation: For large dimensions, dot products grow large, pushing softmax into saturation regions (tiny gradients).

Used in: Transformers (next lecture)

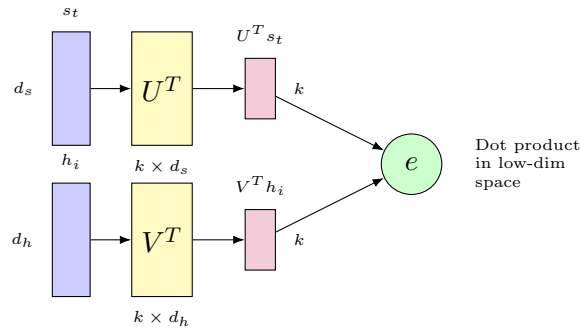
5.4 Reduced-Rank Multiplicative Attention

Definition 5.4 (Low-Rank Bilinear Attention). Factor $W = UV^T$ to reduce parameters:

$$\text{score}(s_t, h_i) = s_t^T UV^T h_i = (U^T s_t)^T (V^T h_i) \quad (17)$$

where $U \in \mathbb{R}^{d_s \times k}, V \in \mathbb{R}^{d_h \times k}$, and $k \ll d_s, d_h$.

Interpretation: Project both vectors to lower-dimensional space, then dot product.



Advantages:

- Far fewer parameters: $k(d_s + d_h)$ vs. $d_s \times d_h$
- Still more expressive than plain dot product

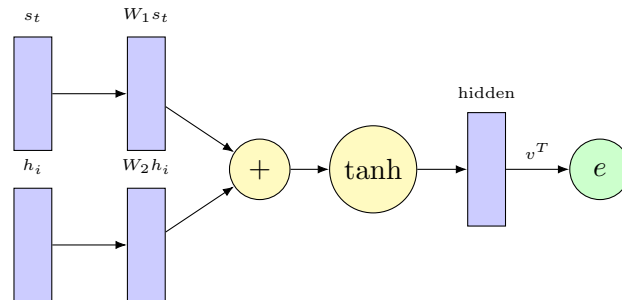
Used in: Transformers use this idea (query/key projections)

5.5 Additive Attention

Definition 5.5 (Additive Attention (Bahdanau et al., 2015)). Use a small feedforward network:

$$\text{score}(s_t, h_i) = v^T \tanh(W_1 s_t + W_2 h_i) \quad (18)$$

where W_1, W_2 are weight matrices and v is a weight vector.



Advantages:

- Most expressive (neural network)
- Original attention formulation
- Empirically strong with proper tuning

Disadvantages:

- Most parameters and computation
- Slower than multiplicative methods

5.6 Comparison of Attention Variants

Modern Practice

- **Transformers use:** Scaled dot product with learned projections (low-rank multiplicative)
- **Best empirical:** Additive (with proper hyperparameter tuning)
- **Most popular:** Multiplicative variants (speed/performance trade-off)

Method	Parameters	Speed	Expressiveness
Dot Product	0	Fastest	Low
Scaled Dot Product	0	Fastest	Low
Multiplicative	$O(d_s \times d_h)$	Fast	Medium
Reduced-Rank	$O(k(d_s + d_h))$	Fast	Medium
Additive	$O(d_s + d_h + d)$	Slow	High

Table 2: Comparison of attention score functions

6 Benefits of Attention

6.1 1. Improved NMT Performance

Empirical Results:

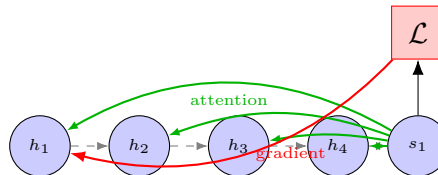
- Bahdanau et al. (2015): Attention improved BLEU by 5–7 points
- Essential for long sentences
- Virtually all modern NMT uses attention

6.2 2. Solves Bottleneck Problem

- No longer compress entire source into one vector
- Decoder accesses full encoder representation
- Information available where and when needed

6.3 3. Helps Vanishing Gradient

- Provides shortcut connections to all encoder states
- Gradient can flow directly from output to any input
- Similar benefit to residual connections



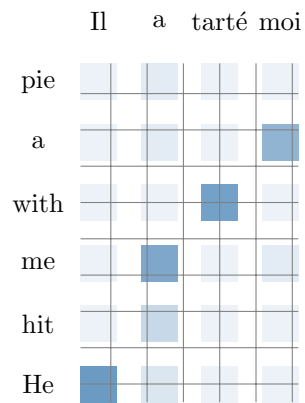
Direct path from loss
to any encoder state

6.4 4. More Human-Like

- Humans look back at source while translating
- Attention allows model to do the same
- More cognitively plausible architecture

6.5 5. Provides Interpretability

Attention weights $\alpha_{t,i}$ show which source words the model attends to when generating each target word.



Darker = higher attention weight

We can see the model learns approximate word alignments without explicit supervision!

7 Attention as a General Tool

7.1 Beyond Machine Translation

Attention is not specific to translation—it's a general neural network building block.

General Attention Framework

Whenever you have:

- A set of **values** (representations)
- A **query** (what you're trying to compute)

You can use attention to:

- Compute relevance of each value to the query
- Take weighted combination of values
- Get query-specific representation

7.2 Applications of Attention

1. Question Answering:

- Query: question representation
- Values: passage representations
- Attend to relevant parts of passage

2. Image Captioning:

- Query: decoder state (caption generation)
- Values: image region features
- Attend to relevant image regions per word

3. Speech Recognition:

- Query: decoder state
- Values: acoustic feature frames
- Attend to relevant audio segments

4. **Summarization:**

- **Query:** summary decoder state
- **Values:** source document sentences
- **Attend** to salient content

5. **Self-Attention (Transformers):**

- **Query:** each position's representation
- **Values:** all positions' representations
- **Attend** within same sequence

7.3 **Why Attention Works So Well**

- **Dynamic routing:** Information flows where it's needed
- **Differentiable:** Can train end-to-end
- **Flexible:** Works with variable-length inputs
- **Interpretable:** Attention weights show what model uses
- **Gradient flow:** Provides shortcuts for backpropagation

8 Summary and Key Takeaways

Key Concepts

1. BLEU Score:

- Automatic MT evaluation via n-gram overlap
- Scores 0–100; higher is better
- Imperfect but practical

2. Bottleneck Problem:

- Vanilla seq2seq compresses everything into one vector
- Information loss, especially for long sentences
- Not human-like

3. Attention Mechanism:

- Decoder can "look back" at all encoder states
- Compute attention scores → softmax → weighted sum
- Different score functions: dot product, multiplicative, additive

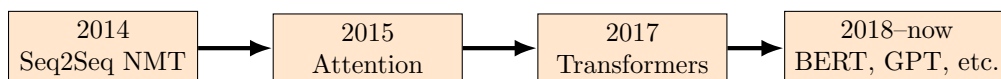
4. Benefits:

- Dramatically improves NMT performance
- Solves bottleneck and vanishing gradient
- Provides interpretability
- General tool beyond MT

5. Looking Ahead:

- Self-attention: attending within same sequence
- Transformers: attention-only architecture (next lecture)
- Foundation of modern NLP (BERT, GPT, etc.)

8.1 Historical Impact



Attention:
Foundational innovation
for modern NLP

9 Further Reading

9.1 Foundational Papers

1. BLEU:

Papineni, K., et al. (2002). BLEU: a method for automatic evaluation of machine translation. *ACL*.

2. Neural MT:

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *NeurIPS*.

3. **Attention (Additive):**

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR*.

4. **Attention (Multiplicative):**

Luong, M. T., Pham, H., & Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. *EMNLP*.

5. **Transformers (Preview):**

Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.

9.2 Additional Resources

- Stanford CS224N Assignment 3: Implementing attention for NMT
- *Attention and Augmented Recurrent Neural Networks* (Distill.pub)
- <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq>